빅데이터기초 파이썬
중간고사 요약본

챕터별 정리

01 파이썬 3주차 핵심 정리

02 파이썬 4주차 핵심정리 - 리스트, 튜플

03 파이썬 5주차 핵심정리 - 딕셔너리, 집합

04 파이썬 데이터 구조 4~5주차 완벽 정리 메모

05 6주차 NumPy 다차원 배열 객체 전체 완벽 요약

06 [7장] 사용자 정의 함수 및 제어 구문 핵심 요약

CHAPTER 01

파이썬 3주차 핵심 정리

1. 변수명 규칙 & 식별자

- 숫자로 시작 불가 (예: 1st_student -> SyntaxError)
- 예약어 사용 불가 (if, for, class, def, return, True, False 등)
- 대소문자 구분 (Data != data)

2. 숫자형 연산자 핵심

- `/` : 일반 나누기 (결과는 항상 float 실수)
- `//` : 버림 나누기 (정수 몫만 반환)
- `%` : 나머지 연산자
- `**` : 거듭제곱 (예: 3 ** 4 = 81)

> **#f-string에서 % 포맷 옵션은 숫자에 100을 곱해 백분율값(12.56%)으로 바꾼뒤, .1지정자에 따라 소수점 첫째자리로 반올림하여 '12.6%'를 출력한다!!! ex) print(f"{rate:.1%}")**

3. 문자열(str) 특성 & 인덱싱/슬라이싱

- 불변성(Immutable): 특정 위치의 문자를 직접 변경 불가 (s[0] = 'a' -> TypeError오류가 발생한다!!!!)
- 슬라이싱: s[start:end:step] (end-1 인덱스까지 추출)
- 문자열 뒤집기: s[::-1]
- 단일인덱싱 접근시 범위를 벗어나면 IndexError가 발생하지만 슬라이싱은 범위를 벗어나도 안전하게 존재하는 범위까지만 추출하고 오류가 나지 않는다!! ex)s = python , print(s[1:100]) : 정상

4. 주요 문자열 함수 & 메서드

- `len(s)` : 전체 길이(공백 포함) 반환 (파이썬 내장함수라 len(text)로 표현한다!!!!!
  (나머지 문자열 함수들은 str에서만 의미있다. => text.split(",")으로 표현
- `split(sep,maxsplit)` : 구분자 기준 분할 -> 리스트 반환 (기본값: 공백)
  (두번째 인자인 maxsplit은 최대분할 횟수를 지정!!!!!!)
- `join(iterable)` : 리스트 원소들을 특정 문자열로 결합
- `replace(old, new, [count])` : old를 new로 교체 (원본은 변경 안 됨)
- `find(sub) / rfind(sub)` : sub의 첫 / 마지막 위치 인덱스 반환 (찾을수 없는경우에는 -1을 반환한다!!)
- `count(sub)` : sub의 출현 횟수 반환
- `upper() / lower()` : 대문자 / 소문자 변환
- `capitalize() / title()` : 첫 글자만 대문자,나머지는 모두 소문자 / 단어별 첫 글자 대문자
- `strip()` : 양쪽공백 제거함수

5. TypeError vs ValueError 구분 (9번)

TypeError: 타입 자체가 안 맞을 때 (예: "age: " + 20 ➔ 문자열과 숫자를 더함)

ValueError: 타입은 맞는데 내부 값의 형태가 잘못됐을 때 (예: int("100a") ➔ 문자열을 정수로 바꾸려는데 'a'가 들어있음)

6. 문자열 곱셈과 print() 공백 (10번)

"abc" * 0 은 빈 문자열("")이 된다.

print("", 0) 을 실행하면 빈 문자열과 0 사이를 print 기본 구분자(공백)가 채우기 때문에 0 (앞에 공백 하나)이 출력됩니다.

CHAPTER 02

파이썬 4주차 핵심정리 - 리스트, 튜플

<파이썬 리스트(List) 핵심 정리>

1. 리스트 기본 특징

- 가변(Mutable) 자료형: 요소 추가, 수정, 삭제 가능
- 대괄호 [] 로 생성하며, 다양한 자료형을 혼합하여 포함 가능
- 인덱싱/슬라이싱 지원 (음수 인덱스 가능, -1은 마지막 요소)
- 빈 리스트 생성: [] 또는 list()

2. 리스트 전용 메서드 (list.method() 형태)

- `.append(x)` : 리스트 맨 끝에 요소 x 하나를 통째로 추가(통채로 하나로 추가함에 주의할것!)(반환값 None!!)
- `.insert(i, x)` : 인덱스 i 위치에 x 추가 (범위 초과 인덱스 지정 시 맨 뒤에 추가)
- `.extend(lst)` : 다른 리스트/시퀀스를 풀어헤쳐 병합 (+= 연산과 동일)
- `.remove(val)` : 리스트에서 값 val을 찾아 첫 번째로 만나는 요소만 삭제 (반환값 없음)
- `.pop(i)` : 인덱스 i 위치의 요소를 삭제하면서 '삭제된 값 반환' (i 생략 시 맨 뒤 요소)
- `.index(val)` : 값 val이 위치한 첫 번째 인덱스 반환 (없으면 ValueError)
- `.count(val)` : 값 val이 출현한 횟수 반환
- `.sort()` : 원본 리스트 자체를 정렬 (반환값은 None) / 내림차순: sort(reverse=True)

3. 범용 내장 함수 및 키워드 (func(list) / keyword 형태)

- `len(lst)` : 리스트 전체 요소 개수(길이) 반환
- `del lst[i]` : 키워드 명령어로 인덱스 i 위치 요소 삭제 (슬라이스 범위 삭제 및 변수 삭제 가능)
- `sorted(lst)` : 원본은 유지하고, 정렬된 '새로운 리스트'를 생성하여 반환
- `in / not in` : 특정 값이 리스트 안에 존재하는지 여부를 True/False로 반환

4. 시험 단골 주의사항

- lst.sort()의 결과값을 print하면 None이 출력됨 (리턴값이 None임!! , 원본 변경) (주의!!!!!!!!!!!!)
- sorted(lst)는 원본 변경 없이 정렬된 새 리스트 반환
- lst[i] 단일 인덱싱은 범위 초과 시 IndexError 발생하지만,
  lst[1:100] 슬라이싱은 범위 초과 시에도 에러 없이 존재하는 범위까지만 안전하게 추출됨

<파이썬 리스트 심화 함정 & 오답 노트>

1. 리스트 제자리 수정 메서드의 반환값 (reverse, sort)

- .reverse(), .sort()는 원본 리스트를 직접 수정(In-place)함
- 두 메서드의 반환값은 항상 None!
  - res = lst.reverse() -> res는 None, lst는 반대로 뒤집힘
  - res = lst.sort() -> res는 None, lst는 정렬됨

2. 대입 연산자(+) vs 제자리 연산자(+=, extend)의 객체 생성 차이

- `a = a + [4, 5]` : 원본을 건드리지 않고 '새로운 리스트 객체'를 생성해 a에 재할당
  - > b = a 선언 후 실행 시, b는 기존 원본[1, 2, 3]을 유지함
- `a += [4, 5] / a.extend([4, 5])` : 기존 리스트 객체를 직접 수정
  - > b = a 선언 후 실행 시, b도 똑같이 [1, 2, 3, 4, 5]로 수정됨

3. 슬라이스 대입 vs 단일 인덱스 대입 차이

- 단일 인덱스 대입: lst[1] = [99]
  - > 지정 위치 원소를 [99] 리스트로 교체 (중첩 리스트 생성: [10, [99], 30])
- 슬라이스 대입: lst[1:4] = [99]
  - > 지정 '범위 구역 전체'를 오른쪽 리스트의 원소들로 치환 (중첩X, [10, 99, 50])

4. 곱셈 연산자(*)로 2차원 리스트 생성 시 참조 공유 함정

- matrix = [[0] * 2] * 3
  - > 똑같은 리스트의 메모리 주소를 3번 복사하여 할당함
  - > matrix[0][0] = 1 변경 시, 모든 행이 함께 변경되어 [[1, 0], [1, 0], [1, 0]]이 됨

5. 얕은 복사(Shallow Copy, .copy() / [:])의 한계

- a.copy() 또는 a[:]는 1차원 리스트 껍데기만 새로 복사함
- 2차원 이상의 중첩 리스트([3, 4])가 내부에 포함되어 있다면, 내부 리스트의 주소는 그대로 공유됨
  - > b[2][0] = 99 로 수정 시 원본 a[2][0]도 99로 변경됨

6. 슬라이싱 음수 Step(증감값) 동작

- `lst[::-1]` : 맨 뒤부터 거꾸로 1칸씩 진행 (리스트 반전)
- `lst[::-2]` : 맨 뒤(마지막 원소)부터 시작하여 거꾸로 2칸씩 건너뜀
  - lst = [1, 2, 3, 4, 5] -> lst[::-2] 결과는 [5, 3, 1]

[파이썬 튜플(Tuple) 핵심 정리 & 시험 출제 포인트]

1. 튜플의 가장 중요한 특징

- 불변성(Immutable): 한 번 생성된 튜플의 요소는 수정, 추가, 삭제가 불가능함
  (튜플의 불변성이란 "원소들의 메모리 주소(참조)를 다른 주소로 바꿀 수 없다"는 뜻이기때문에 원소로 리스트 들어올수있다)
- 리스트에 비해 메모리 사용이 적고 처리 속도가 빠름

2. 튜플 생성 문법 & 원소 1개일 때 주의점

- 기본 생성: t = (1, 2, 3) 또는 소괄호 생략 t = 1, 2, 3
- 원소가 1개일 때 필수 표기법:
  - t = (10) -> 괄호로 묶인 정수(int)로 인식!
  - t = (10,) -> 원소 뒤에 콤마(,)를 반드시 붙여야 튜플(tuple)이 됨!
- 빈 튜플: t = () 또는 t = tuple()

3. 튜플에서 가능한 연산 vs 불가능한 연산

- [가능한 연산] (조회 및 새 객체 생성)
  - 인덱싱 / 슬라이싱 (t[0], t[1:3])
  - 덧셈 / 곱셈 연산 (t1 + t2, t * 3) -> 기존 튜플을 바꾸는 게 아니라 '새로운 튜플'을 생성
  - len(t), in / not in 연산
  - .count(val) (개수 세기), .index(val) (인덱스 위치 찾기)
- [불가능한 연산] (수정 시 TypeError 발생!)
  - t[0] = 99 (요소 변경 불가)
  - del t[0] (특정 요소 삭제 불가)
  - append(), extend(), insert(), pop(), remove(), sort() 등 수정 메서드 사용 불가

4. 튜플 패킹(Packing) & 언패킹(Unpacking)

- 패킹: 여러 값을 하나의 튜플로 묶는 것 (t = 10, 20, 30)
- 언패킹: 튜플의 요소를 여러 변수에 나누어 담는 것 (a, b, c = t)
- 변수 값 교환 (x, y = y, x):
  - 우변 (y, x)에서 임시 튜플이 생성되어 패킹된 후, 좌변 (x, y)로 동시에 언패킹되어 교환됨 (임시변수 temp 불필요)

5. 가변 언패킹 (* 연산자)

- a, *b, c = (1, 2, 3, 4, 5)
  - a = 1, c = 5 가 들어가고, 남은 (2, 3, 4)는 *b에 할당됨
  - 주의: * 연산자로 묶인 변수 b의 타입은 튜플이 아니라 '리스트([2, 3, 4])'가 됨!

CHAPTER 03

파이썬 5주차 핵심정리 - 딕셔너리, 집합

<딕셔너리(Dictionary) 핵심 정리 & 시험 출제 포인트>

1. 딕셔너리의 기본 구조 & 특징

- 키(Key)와 값(Value)의 쌍(Pair)으로 데이터를 저장하는 가변(Mutable) 데이터 구조
- 중괄호({}) 또는 dict() 함수를 이용해 생성
  - 예: student = {"name": "Alice", "age": 21, "major": "CS"}

2. Key(키) 지정 시 필수 규칙 (★ 시험 단골 트릭)

- Key는 중복될 수 없음 (중복 지정 시 나중에 작성한 값으로 덮어씌워짐)
- Key는 반드시 '불변(Immutable) 객체'만 사용 가능!
  - Key 가능: int, float, str, tuple
  - Key 불가능 (TypeError 발생): list, dict, set

3. Key 접근 및 조회 방식의 차이 (★ 필수 구분)

- `dict['key']` : 존재하지 않는 키로 접근 시 KeyError 에러 발생
- `dict.get('key')` : 존재하지 않는 키로 접근해도 에러 없이 None 반환
  - `dict.get('key', default_val)` : 키가 없으면 지정한 기본값을 반환

4. 요소 추가, 수정 및 삭제

- 추가/수정:
  - dict['new_key'] = value
  - update() 메서드: 다른 딕셔너리나 (Key, Value) 쌍을 병합/수정 (반환값은 None)
- 삭제:
  - `del dict['key']` : 해당 키-값 쌍 삭제 (키가 없으면 KeyError)
  - `dict.pop('key')` : 해당 키의 값을 반환하고 삭제 (키가 없으면 지정한 기본값을 반환)
  - `dict.clear()` : 딕셔너리의 모든 요소 삭제 (빈 딕셔너리 {} 생성)

5. items(), keys(), values() 함수와 list() 변환

- `dict.keys()` : 모든 Key를 dict_keys 객체로 반환
- `dict.values()` : 모든 Value를 dict_values 객체로 반환
- `dict.items()` : 모든 (Key, Value) 튜플 쌍을 dict_items 객체로 반환
- 주의점: dict_items, dict_keys, dict_values는 '뷰(View) 객체'이므로
  for문 반복은 가능하지만 인덱싱(e.g. dict.items()[0])은 불가능함!
  - > 인덱스 접근이나 리스트 메서드를 쓰려면 list(dict.items()) 형태로 변환 필수

<집합(Set) 핵심 정리 & 시험 출제 포인트>

1. 집합의 기본 특징

- 값(value)은 없고 키(key)만 남은 딕셔너리와 유사!!!!
- 중복을 허용하지 않음 (자동 중복 제거)
- 순서가 없음 (Unordered) ➔ 인덱싱(s[0]) 및 슬라이싱(s[1:3]) 절대 불가 (TypeError 발생)
- 중괄호({}) 또는 set() 함수로 생성

2. 빈 집합(Empty Set) 생성 주의사항 (★ 시험 단골 트릭)

- s = {} ➔ 빈 딕셔너리(dict)가 생성됨!
- s = set() ➔ 반드시 set() 함수를 사용해야 '빈 집합'이 생성됨!!!!

3. 집합의 원소 조건 (Unhashable Type)

- 집합의 원소는 반드시 '불변(Immutable) 객체'만 가능!
  - 원소 가능: int, float, str, tuple
  - 원소 불가능 (TypeError): list, dict, set

4. 원소 추가 및 삭제 메서드

- 추가:
  - `s.add(x)` : 단일 요소 x 추가
  - `s.update(seq)` : 여러 요소(리스트, 튜플 등)를 한 번에 추가 (In-place 수정, 반환값 None)
- 삭제:
  - `s.remove(x)` : 원소 x 삭제 (원소가 없으면 KeyError 발생!)
  - `s.discard(x)` : 원소 x 삭제 (원소가 없어도 에러 없이 안전하게 무시)
  - `s.pop()` : 임의의(아무) 원소를 하나 제거하고 반환 (빈 집합이면 KeyError)
  - `s.clear()` : 모든 원소 삭제 (빈 집합 set()으로 만듦)

5. 집합 연산자 & 메서드

- `합집합 (A | B / A.union(B))` : A와 B의 모든 원소
- `교집합 (A & B / A.intersection(B))` : A와 B의 공통 원소
- `차집합 (A - B / A.difference(B))` : A에만 있는 원소
- 대칭 차집합 (A ^ B / A.symmetric_difference(B)): A와 B 중 한 곳에만 있는 원소 (합집합 - 교집합)

> **#집합(Set)은 내부적으로 해시 테이블(Hash Table) 구조를 사용하기 때문에 모든 원소가 불변(Immutable) 객체여야 한다. (list, dict, set과 같이 원소를 수정할 수 있는 가변 객체는 해시값을 만들 수 없어 집합의 원소로 사용 시 TypeError발생)**

> **#파이썬에서 1 == 1.0 == True는 모두 True(같은 값)로 평가되기 때문에, 집합을 만들 때 1, 1.0, True는 서로 동일한 값으로 취급되어 중복이 제거되고 맨 처음 등장한 1만 남는다.**

[관계 판단 메서드 (True / False)]

- `A.issubset(B)` : A <= B (A가 B의 부분집합인가?)
- `A.issuperset(B)` : A >= B (A가 B의 상위집합인가?)
- `A.isdisjoint(B)` : A와 B에 공통 원소가 하나도 없는가? (서로소 집합)

CHAPTER 04

파이썬 데이터 구조 4~5주차 완벽 정리 메모

1. 자료구조별 가변성(Mutability) 및 특징

- `리스트(list)` : 가변(Mutable) | [1, 2] | 인덱싱/슬라이싱 가능
- `튜플(tuple)` : 불변(Immutable) | (1, 2) | 인덱싱 가능 / 수정·추가·삭제 불가
- 딕셔너리(dict): 가변(Mutable) | {'a': 1} | Key-Value / Key는 불변만 가능
- `집합(set)` : 가변(Mutable) | {1, 2} | 중복 불가 / 순서 없음(인덱싱 불가)

> ※ **튜플의 불변성 핵심:**
> - 튜플 내부 리스트의 '내부 원소 변경(tup[2].append())'은 가능 (참조 주소만 고정되기 때문)
> - 단, 리스트를 포함한 튜플은 해시 불가능해져 Dict Key / Set 원소로 사용 불가!

2. 유사 기능 메서드 비교 (★ 중간고사 핵심 포인트)

[원소 추가]

- `list.append(x)` : 리스트 맨 뒤에 '단일 원소' x 추가
- `list.extend(seq)` : 리스트에 '여러 원소(iterable)'를 풀어 연결 (list += seq)
- `set.add(x)` : 집합에 '단일 원소' x 추가
- `set.update(seq)` : 집합에 '여러 원소(iterable)'를 풀어 추가 (In-place, 반환값 None)
- `dict.update(d2)` : 딕셔너리에 키-값 쌍 병합/수정 (In-place, 반환값 None)

[원소 삭제]

- `set.remove(x)` : 원소 x 삭제 ➔ 없으면 KeyError 발생! (위험)
- `set.discard(x)` : 원소 x 삭제 ➔ 없어도 에러 없이 무시 (안전)
- `del d[key]` : Key 삭제 ➔ 키 없으면 KeyError
- `d.pop(key, def)` : Key 삭제 후 Value 반환 ➔ 기본값(def) 설정 시 KeyError 방지

[조회 / 탐색]

- `d[key]` : 대괄호 접근 ➔ 키 없으면 KeyError 발생!
- `d.get(key, def)` : 메서드 접근 ➔ 키 없으면 None 또는 기본값(def) 반환!
- `"x" in dict` : 딕셔너리의 'Key'만 탐색! (Value 탐색 X, O(1) 속도)
- `"x" in set` : 집합의 '원소' 탐색! (O(1) 속도)

[정렬]

- `list.sort()` : 리스트 자체를 직접 수정 (In-place) ➔ 반환값은 항상 None!
- `sorted(seq)` : 원본은 유지, 정렬된 '새로운 리스트' 반환!

3. 집합(Set) 전용 메서드 정리

[생성]

- `s = {}` : ✘ 빈 딕셔너리(dict) 생성!
- `s = set()` : ◯ 빈 집합(set) 생성!

[집합 연산]

- `합집합` : A | B / A.union(B)
- `교집합` : A & B / A.intersection(B)
- `차집합` : A - B / A.difference(B)
- 대칭차집합: A ^ B / A.symmetric_difference(B)

[관계 판단 (True / False)]

- `A.issubset(B)` : A <= B (A가 B의 부분집합인가?)
- `A.issuperset(B)` : A >= B (A가 B의 상위집합인가?)
- `A.isdisjoint(B)` : A와 B에 공통 원소가 하나도 없는가? (서로소)

4. 주요 예외(Error) 발생 패턴

- `KeyError` : Dict/Set 없는 Key나 원소 삭제/조회 (d["x"], set.remove("x"))
- `TypeError` : Set/Dict Key에 가변 객체(List) 사용, Set/View 객체에 인덱싱(s[0])
- AttributeError: 튜플에 append 사용 등 없는 메서드 호출,
  또는 None 객체에 메서드 호출 (res = lst.sort() 후 res.append())

CHAPTER 05

6주차 NumPy 다차원 배열 객체 전체 완벽 요약

1. NumPy 개요 & 특징

- `NumPy(Numerical Python)` : 수치 계산 및 대용량 데이터 처리를 위한 핵심 패키지
- C언어 기반의 연속된 메모리 배치와 Vectorization(벡터화) 연산으로 파이썬 기본 리스트보다 연산 속도가 월등히 빠름 (약 20~30배)
- 리스트와 달리 모든 원소가 '동일한 데이터 타입(Homogeneous)'이어야 함 (타입 혼용 시 상위 타입으로 자동 변환)

2. ndarray 4대 핵심 속성 (★ 시험 필수)

- `arr.ndim` : 차원 수 (1차원: 1, 2차원: 2)
- `arr.shape` : 각 차원의 크기 (튜플 형태, 예: (2, 4))
- `arr.size` : 전체 원소의 총 개수 (shape 내부 값들의 곱)
- `arr.dtype` : 원소의 데이터 타입 (int64, float64 등)
  ※ 주의: type(arr)는 <class 'numpy.ndarray'>, arr.dtype은 원소의 타입!

3. 주요 배열 생성 & 난수 생성 함수

[기본 생성]

- `np.array(list/tuple)` : 리스트/튜플을 ndarray로 변환
- `np.zeros(shape)` : 0으로 채워진 배열 생성 (기본 float64 -> 0.)
- `np.ones(shape)` : 1로 채워진 배열 생성 (기본 float64 -> 1.)
- `np.full(shape, val)` : 지정한 값(val)으로 채워진 배열 생성
- `np.arange(start, stop, step)` : range와 유사한 ndarray 생성
- `np.linspace(start, stop, num)` : 일정 간격으로 분할된 배열 생성
- `np.eye(N)` : N x N 크기의 단위행렬 생성

[난수 생성 (np.random)]

- `seed()` : 난수 고정
- `rand()` : [0, 1) 균등분포 표본
- `randint()` : 범위 내 임의의 정수 추출
- `randn()` : 평균 0, 표준편차 1인 표준정규분포 표본
- `normal()` : 가우시안 정규분포 표본

4. 인덱싱 & 슬라이싱

- `파이썬 기본 리스트` : arr[0][0] 형태 (대괄호 2개)
- `NumPy 다차원 배열` : arr[0, 0] 형태 권장 (대괄호 1개 + 콤마, C언어 방식 텔레포트 접근)
- `슬라이싱` : arr[행_start:행_stop, 열_start:열_stop] 형태로 한번에 범위 지정

5. 구조 변경(reshape), 전치(T), 연결 & 분할

- `arr.reshape(r, c)` : 원소 개수 유지하며 모양 변경 (원소 총개수 불일치 시 ValueError)
- `arr.T / arr.transpose()` : 행과 열을 뒤바꿈 (데이터 복사 없이 뷰(View) 반환)

[배열 연결]

- `np.concatenate([x, y], axis=0)` : 세로(위아래) 연결
- `np.concatenate([x, y], axis=1)` : 가로(좌우) 연결
- `np.vstack([x, y])` : Vertical stack (세로/수직 쌓기)
- `np.hstack([x, y])` : Horizontal stack (가로/수평 붙이기)

[배열 분할]

- `np.split(z, [idx], axis=0) / np.vsplit()` : 세로 축 따라 가로로 자름 (위/아래 분할)
- `np.split(z, [idx], axis=1) / np.hsplit()` : 가로 축 따라 세로로 자름 (좌/우 분할)

6. 유니버설 함수 (UFuncs)

- 배열의 각 원소 단위로 빠르고 일괄된 연산을 수행하는 함수
- `단항 UFunc` : 인자 1개 (np.abs, np.sqrt, np.exp 등)
- `이항 UFunc` : 인자 2개 (np.add, np.maximum, np.subtract 등)

7. 브로드캐스팅 (Broadcasting) 3대 규칙 (★ 최다 출제)

크기가 다른 배열 간 이항 연산 시 자동으로 크기를 맞춰주는 규칙

규칙 1: 두 배열의 차원 수가 다르면, 작은 배열의 '앞쪽(왼쪽)'에 1을 채움

규칙 2: 차원의 크기가 다를 때, 크기가 1인 차원은 상대방 크기에 맞게 복사/확장

규칙 3: 두 차원의 크기가 다르고 둘 다 1이 아니면 ➔ ValueError 발생!

- (2, 3) + (3,) ➔ (3,)가 (1, 3)이 됨 ➔ (2, 3) + (1, 3) ➔ (2, 3)으로 연산 가능!
- (3, 2) + (3,) ➔ (3,)가 (1, 3)이 됨 ➔ 뒤쪽 차원 2 != 3 이므로 ValueError!

CHAPTER 06

[7장] 사용자 정의 함수 및 제어 구문 핵심 요약

1. 사용자 정의 함수 (User-Defined Functions)

- 기본 구조: def 함수명(매개변수): ... return 결과
- 인자 전달 방식:
  - 위치 인자: 순서대로 매개변수에 대응
  - 키워드 인자: func(x1=1, x2=2) 처럼 지정 (순서 바뀌어도 OK)
  - 혼용 규칙: 위치 인자가 반드시 키워드 인자보다 먼저 나와야 함
  - 기본값 매개변수: def func(a, b=3) 형태 (기본값 매개변수는 항상 뒤쪽에 위치)
- 가변 인자 취합 (* / **):
  - *args: 위치 인자들을 '튜플(tuple)'로 묶음
  - **kwargs: 키워드 인자들을 '딕셔너리(dict)'로 묶음
- 익명 함수 (lambda):
  - lambda 매개변수: 표현식
  - 예: students.sort(key=lambda x: x[1]) (정렬 기준 지정 시 활용)

2. 조건문 (Conditionals)

- 기본 구조: if - elif - else
- 입력값 검증 logic (is_even 예제):
  - `x % 1 == 0` : 소수점 없는 '정수'인지 검사
  - `x > 0` : '양수'인지 검사
  - `x % 2 == 0` : 정수/양수 조건을 만족하는 값만 짝수/홀수 판별
  - 실수(3.5 등)나 0 이하 값은 "NA" 처리

3. 반복문 및 순회 유틸리티 (Loops)

- `range(start, stop)` : start부터 (stop - 1)까지만 생성 (끝값 미포함)
  - 예: list(range(1, 8)) -> [1, 2, 3, 4, 5, 6, 7] (총 7개)
- 반복 제어:
  - continue: 현재 회차 스킵 후 다음 반복 진행
  - break: 반복문 전체 즉시 종료
- 주요 순회 함수:
  - `enumerate(iterable)` : (인덱스, 값) 튜플 쌍 반환
    예: for i, val in enumerate(fruits):
  - `zip(list1, list2)` : 여러 리스트의 같은 인덱스끼리 1:1로 묶음 (가장 짧은 리스트 길이 기준 종료)